Human Verification: `Containers.Cont`

The Category of Containers: Identity, Composition, and Finite (Co)Products

Machine-generated from `lean/Cont.lean` for review by Neil Ghani

July 2026

Abstract

This document presents every definition and theorem *statement* from the Lean 4 module `Containers.Cont` in three parallel forms: the original Lean, a readable Haskell rendering, and precise mathematical English. Proofs are omitted — they are machine-checked by Lean. The purpose is to let a human verifier confirm that the formalisation captures the intended mathematics.

Contents

1 Overview **2**

2 Identity and composition **2**
2.1 Definition: `ContainerMorphism.id` . . . 2
2.2 Definition: `ContainerMorphism.comp` . . . 2

3 Extensionality **3**
3.1 Theorem: `ContainerMorphism.ext` . . . 3
3.2 Theorem: `ContainerMorphism.ext'` . . . 3

4 Category laws **4**
4.1 Theorem: `ContainerMorphism.id_comp` . . . 4
4.2 Theorem: `ContainerMorphism.comp_id` . . . 4
4.3 Theorem: `ContainerMorphism.comp_assoc` . . . 4

5 The `Category` typeclass and the `Cont` instance **5**
5.1 Structure: `Category` . . . 5
5.2 Instance: `Cont` . . . 5

6 Binary coproducts in `Cont` **6**
6.1 Definition: `Container.coprod` . . . 6
6.2 Definition: `Container.inl` . . . 6
6.3 Definition: `Container.inr` . . . 6
6.4 Definition: `ContainerMorphism.desc` . . . 7
6.5 Theorem: `ContainerMorphism.inl_desc` . . . 7
6.6 Theorem: `ContainerMorphism.inr_desc` . . . 7
6.7 Theorem: `ContainerMorphism.desc_eta` . . . 7
6.8 Theorem: `ContainerMorphism.desc_unique` . . . 8

7 Binary products in `Cont` **8**
7.1 Definition: `Container.prod` . . . 8
7.2 Definition: `Container.fst` . . . 9

7.3 Definition: `Container.snd` . . . 9
7.4 Definition: `ContainerMorphism.pair` . . . 9
7.5 Theorem: `ContainerMorphism.pair_fst` . . . 10
7.6 Theorem: `ContainerMorphism.pair_snd` . . . 10
7.7 Theorem: `ContainerMorphism.pair_eta` . . . 10
7.8 Theorem: `ContainerMorphism.pair_unique` . . . 11

8 Translation conventions **11**

1 Overview

This file builds on `Containers.Basic` to equip containers and their morphisms with the structure of a *category* **Cont**.

- **Objects** are containers $C = (S \triangleleft P)$.
- **Morphisms** $(S \triangleleft P) \to (S' \triangleleft P')$ are container morphisms: a forward shape map $f : S \to S'$ together with a *contravariant* position map $f_s^\sharp : P'(f(s)) \to P(s)$ for each s.

The key insight is that the contravariance of the position map makes all three category laws hold *definitionally* (by `rfl`) — no transports are needed. This contrasts sharply with the directed-container file, where genuine dependent transports appear.

The file also establishes binary coproducts and products in **Cont**, completing the algebra of container constructors.

2 Identity and composition

2.1 Definition: `ContainerMorphism.id`

Definition 2.1 (Identity morphism)**.** The **identity** container morphism on $C = (S \triangleleft P)$ is the pair $(\mathrm{id}_S, \mathrm{id}_{P(s)})$: identity on shapes and identity on positions.

Lean.

```
def ContainerMorphism.id (C : Container) : ContainerMorphism C C where
  onShapes := id
  onPos := fun _ ⇒ id
```

Haskell.

```
idMorph :: Morph s p s p
idMorph = Morph id (\_ → id)
```

2.2 Definition: `ContainerMorphism.comp`

Definition 2.2 (Composition of container morphisms)**.** Given morphisms $\varphi = (u, g) : C \to D$ and $\psi = (v, h) : D \to E$, their **composite** $\psi \circ \varphi : C \to E$ is

$$(\psi \circ \varphi)_{\text{shapes}} \;=\; v \circ u, \qquad (\psi \circ \varphi)_s^\sharp \;=\; g_s \circ h_{u(s)}.$$

Shape maps compose forwards; position maps compose *backwards*, with ψ's pullback applied first, then φ's.

Lean.
```
def ContainerMorphism.comp {C D E : Container}
    (phi : ContainerMorphism C D) (psi : ContainerMorphism D E) :
    ContainerMorphism C E where
  onShapes := fun s ⇒ psi.onShapes (phi.onShapes s)
  onPos := fun s ⇒ phi.onPos s  ∘  psi.onPos (phi.onShapes s)
```

Haskell.
```
compMorph :: Morph s p t q → Morph t q u r → Morph s p u r
compMorph (Morph u g) (Morph v h) =
  Morph (v . u) (\s → g s . h (u s))
```

3 Extensionality

3.1 Theorem: `ContainerMorphism.ext`

Theorem 3.1 (Extensionality (HEq form))**.** Two container morphisms $\varphi, \psi : C \to D$ are equal if their shape maps are equal and their position maps are heterogeneously equal (`HEq`):

$$\varphi.\text{onShapes} = \psi.\text{onShapes} \;\wedge\; \varphi.\text{onPos} \approx \psi.\text{onPos} \;\Longrightarrow\; \varphi = \psi.$$

The heterogeneous equality is necessary because the type of φ.onPos depends on φ.onShapes; after substituting the shape equality, it collapses to ordinary equality.

Lean.
```
theorem ContainerMorphism.ext {C D : Container} :
    ∀ {phi psi : ContainerMorphism C D},
      phi.onShapes = psi.onShapes → HEq phi.onPos psi.onPos → phi =
         psi
```

Haskell.
```
-- In Haskell there is no dependent type on onPos, so ordinary
-- structural equality suffices: equal onShapes and equal onPos
-- implies equal Morph values.
```

Fidelity note. The use of `HEq` (heterogeneous equality) is forced by the dependent typing of `onPos`: its type mentions `onShapes`, so two position maps that live in *different* function types cannot be compared with `=` until the shape equality is substituted. This subtlety disappears in Haskell where position maps are not dependent.

3.2 Theorem: `ContainerMorphism.ext'`

Theorem 3.2 (Extensionality (pointwise form))**.** A more convenient extensionality principle: given that the shape maps agree (`hs` : φ.onShapes = ψ.onShapes), two morphisms are equal whenever their position maps agree after transporting positions along `hs`. When the shape maps agree *definitionally*, the transport is the identity and the hypothesis holds by `rfl`.

Lean.
```
theorem ContainerMorphism.ext' {C D : Container} :
    ∀ {phi psi : ContainerMorphism C D} (hs : phi.onShapes = psi.
       onShapes),
      (∀ s p, phi.onPos s p = psi.onPos s (congrFun hs s ▸ p)) →
         phi = psi
```

Haskell.
```
-- Again, in Haskell ordinary pointwise equality of the fields
   suffices.
-- (equal onShapes and pointwise-equal onPos) ⇒ equal Morph values.
```

Fidelity note. The expression `congrFun hs s ▷ p` transports the position p along the shape equality: it coerces $p : P'(\varphi(s))$ to $P'(\psi(s))$ so that both sides of the position-map equation live in the same type. This is purely a Lean book-keeping device with no Haskell analogue.

4 Category laws

4.1 Theorem: `ContainerMorphism.id_comp`

Theorem 4.1 (Left unit law)**.** For every morphism $\varphi : C \to D$,

$$\mathrm{id}_C \circ \varphi \;=\; \varphi.$$

This holds definitionally (by `rfl`).

Lean.

```
theorem ContainerMorphism.id_comp {C D : Container}
    (phi : ContainerMorphism C D) :
    (ContainerMorphism.id C).comp phi = phi
```

Haskell.

```
-- Left unit law:  compMorph idMorph phi = phi
```

4.2 Theorem: `ContainerMorphism.comp_id`

Theorem 4.2 (Right unit law)**.** For every morphism $\varphi : C \to D$,

$$\varphi \circ \mathrm{id}_D \;=\; \varphi.$$

This holds definitionally (by `rfl`).

Lean.

```
theorem ContainerMorphism.comp_id {C D : Container}
    (phi : ContainerMorphism C D) :
    phi.comp (ContainerMorphism.id D) = phi
```

Haskell.

```
-- Right unit law:  compMorph phi idMorph = phi
```

4.3 Theorem: `ContainerMorphism.comp_assoc`

Theorem 4.3 (Associativity)**.** For every composable triple $\varphi : C \to D$, $\psi : D \to E$, $\chi : E \to F$,

$$(\chi \circ \psi) \circ \varphi \;=\; \chi \circ (\psi \circ \varphi).$$

This holds definitionally (by `rfl`).

Lean.

```
theorem ContainerMorphism.comp_assoc {C D E F : Container}
    (phi : ContainerMorphism C D) (psi : ContainerMorphism D E)
    (chi : ContainerMorphism E F) :
    (phi.comp psi).comp chi = phi.comp (psi.comp chi)
```

Haskell.

```
-- Associativity:
-- compMorph (compMorph phi psi) chi = compMorph phi (compMorph psi
    chi)
```

5 The `Category` typeclass and the `Cont` instance

5.1 Structure: `Category`

Definition 5.1 (Category typeclass)**.** A minimal, Mathlib-free category typeclass on a type $\mathrm{Obj} : \mathbf{Type}_u$. It bundles:

- a Hom-family $\mathrm{Hom} : \mathrm{Obj} \to \mathrm{Obj} \to \mathbf{Type}_v$,
- identity morphisms $\mathrm{id}_X : \mathrm{Hom}(X, X)$,
- composition $\mathrm{comp} : \mathrm{Hom}(X, Y) \to \mathrm{Hom}(Y, Z) \to \mathrm{Hom}(X, Z)$ (diagrammatic order),
- proofs of left unit, right unit, and associativity.

This mirrors `CategoryTheory.Category` from Mathlib but is self-contained.

Lean.

```
class Category (Obj : Type u) where
  Hom : Obj → Obj → Type v
  id  : (X : Obj) → Hom X X
  comp : {X Y Z : Obj} → Hom X Y → Hom Y Z → Hom X Z
  id_comp : ∀ {X Y : Obj} (f : Hom X Y), comp (id X) f = f
  comp_id : ∀ {X Y : Obj} (f : Hom X Y), comp f (id Y) = f
  assoc : ∀ {W X Y Z : Obj} (f : Hom W X) (g : Hom X Y) (h : Hom Y Z),
    comp (comp f g) h = comp f (comp g h)
```

Haskell.

```
class Category hom where
  identity :: hom a a
  compose  :: hom a b → hom b c → hom a c
  -- law: compose identity f = f
  -- law: compose f identity = f
  -- law: compose (compose f g) h = compose f (compose g h)
```

5.2 Instance: `Cont`

Definition 5.2 (The category of containers)**. Cont** is the category whose objects are containers and whose morphisms are container morphisms, with the identity and composition defined above. All three category laws hold definitionally.

Lean.

```
instance Cont : Category Container where
  Hom     := ContainerMorphism
  id      := ContainerMorphism.id
  comp    := ContainerMorphism.comp
  id_comp := ContainerMorphism.id_comp
  comp_id := ContainerMorphism.comp_id
  assoc   := ContainerMorphism.comp_assoc
```

Haskell.

```
-- Container morphisms form a category.
-- Objects: containers (s, p).
-- Morphisms: Morph s p t q with idMorph and compMorph.
-- All laws hold by computation.
```

6 Binary coproducts in `Cont`

The coproduct of $C = (S \triangleleft P)$ and $D = (T \triangleleft Q)$ has shape type $S + T$ and position type given case-wise: an $\mathrm{inl}(s)$-shape has positions $P(s)$; an $\mathrm{inr}(t)$-shape has positions $Q(t)$.

6.1 Definition: `Container.coprod`

Definition 6.1 (Coproduct container)**.** The **coproduct** $C \oplus D$ of containers $C = (S \triangleleft P)$ and $D = (T \triangleleft Q)$ is the container

$$(S + T) \triangleleft [P, Q],$$

where $[P, Q](\mathrm{inl}(s)) = P(s)$ and $[P, Q](\mathrm{inr}(t)) = Q(t)$.

Lean.

```
def Container.coprod (C D : Container) : Container where
  Shape := C.Shape .+. D.Shape      -- Sum type: C.Shape + D.Shape
  Pos   := Sum.elim C.Pos D.Pos
```

Haskell.

```
data CoprodShape s t = InlShape s | InrShape t

coprodPos :: (s → p) → (t → q) → CoprodShape s t → Either p q
coprodPos posC _    (InlShape s) = Left  (posC s)
coprodPos _    posD (InrShape t) = Right (posD t)
-- or simply: Either s t with positions (p `Either` q) case-wise
```

6.2 Definition: `Container.inl`

Definition 6.2 (Left injection)**.** The **left injection** $\iota_C : C \to C \oplus D$ has shape map $\mathrm{inl} : S \to S + T$ and position map the identity (the positions of an $\mathrm{inl}(s)$-shape in $C \oplus D$ are exactly $P(s)$).

Lean.

```
def Container.inl (C D : Container) : ContainerMorphism C (C.coprod D)
    where
  onShapes := Sum.inl
  onPos    := fun _ ⇒ id
```

Haskell.

```
inlMorph :: Morph s p (CoprodShape s t) (Either p q)
inlMorph = Morph InlShape (\_ → Left)
-- onPos sends Left p back to p (identity on Left branch)
```

6.3 Definition: `Container.inr`

Definition 6.3 (Right injection)**.** The **right injection** $\iota_D : D \to C \oplus D$ has shape map $\mathrm{inr} : T \to S + T$ and position map the identity.

Lean.

```
def Container.inr (C D : Container) : ContainerMorphism D (C.coprod D)
    where
  onShapes := Sum.inr
  onPos    := fun _ ⇒ id
```

Haskell.

```
inrMorph :: Morph t q (CoprodShape s t) (Either p q)
inrMorph = Morph InrShape (\_ → Right)
```

6.4 Definition: `ContainerMorphism.desc`

Definition 6.4 (Copairing)**.** Given $\varphi : C \to E$ and $\psi : D \to E$, their **copairing** $[\varphi, \psi] : C \oplus D \to E$ has shape map $[u_\varphi, u_\psi]$ (dispatch on inl/inr) and position map: on an inl(s)-shape use $\varphi^\sharp_s$; on an inr(t)-shape use $\psi^\sharp_t$.

Lean.

```
def ContainerMorphism.desc {C D E : Container}
    (phi : ContainerMorphism C E) (psi : ContainerMorphism D E) :
    ContainerMorphism (C.coprod D) E where
  onShapes := Sum.elim phi.onShapes psi.onShapes
  onPos := fun s ⇒ match s with
    | Sum.inl s ⇒ phi.onPos s
    | Sum.inr s ⇒ psi.onPos s
```

Haskell.

```
desc :: Morph s p u r → Morph t q u r
     → Morph (CoprodShape s t) (Either p q) u r
desc (Morph u g) (Morph v h) = Morph
  (\sh → case sh of { InlShape s → u s; InrShape t → v t })
  (\sh → case sh of { InlShape s → g s; InrShape t → h t })
```

6.5 Theorem: `ContainerMorphism.inl_desc`

Theorem 6.5 (Left computation rule for copairing)**.**

$$\iota_C \circ [\varphi, \psi] \;=\; \varphi.$$

Holds definitionally (`rfl`).

Lean.

```
theorem ContainerMorphism.inl_desc {C D E : Container}
    (phi : ContainerMorphism C E) (psi : ContainerMorphism D E) :
    (Container.inl C D).comp (phi.desc psi) = phi
```

Haskell.

```
-- compMorph inlMorph (desc phi psi) = phi
```

6.6 Theorem: `ContainerMorphism.inr_desc`

Theorem 6.6 (Right computation rule for copairing)**.**

$$\iota_D \circ [\varphi, \psi] \;=\; \psi.$$

Holds definitionally (`rfl`).

Lean.

```
theorem ContainerMorphism.inr_desc {C D E : Container}
    (phi : ContainerMorphism C E) (psi : ContainerMorphism D E) :
    (Container.inr C D).comp (phi.desc psi) = psi
```

Haskell.

```
-- compMorph inrMorph (desc phi psi) = psi
```

6.7 Theorem: `ContainerMorphism.desc_eta`

Theorem 6.7 (Universal property of coproduct (η-rule))**.** Every morphism $\chi : C \oplus D \to E$ is the copairing of its composites with the injections:

$$[\chi \circ \iota_C,\ \chi \circ \iota_D] \;=\; \chi.$$

Together with the two computation rules, this is the full universal property: $\mathrm{Hom}(C \oplus D, E) \cong \mathrm{Hom}(C, E) \times \mathrm{Hom}(D, E)$ naturally in E.

Lean.
```
theorem ContainerMorphism.desc_eta {C D E : Container}
    (chi : ContainerMorphism (C.coprod D) E) :
    ((Container.inl C D).comp chi).desc
      ((Container.inr C D).comp chi) = chi
```

Haskell.
```
-- desc (compMorph inlMorph chi) (compMorph inrMorph chi) = chi
```

6.8 Theorem: `ContainerMorphism.desc_unique`

Theorem 6.8 (Uniqueness of the copairing mediating morphism)**.** For every cocone ($\varphi : C \to E$, $\psi : D \to E$), there is a *unique* morphism $\chi : C \oplus D \to E$ with $\chi \circ \iota_C = \varphi$ and $\chi \circ \iota_D = \psi$, namely $\chi = [\varphi, \psi]$.

Lean.
```
theorem ContainerMorphism.desc_unique {C D E : Container}
    (phi : ContainerMorphism C E) (psi : ContainerMorphism D E)
    (chi : ContainerMorphism (C.coprod D) E)
    (h1 : (Container.inl C D).comp chi = phi)
    (h2 : (Container.inr C D).comp chi = psi) :
    chi = phi.desc psi
```

Haskell.
```
-- If compMorph inlMorph chi = phi and compMorph inrMorph chi = psi
-- then chi = desc phi psi
```

7 Binary products in `Cont`

The product of $C = (S \triangleleft P)$ and $D = (T \triangleleft Q)$ has shape type $S \times T$ and position type $P(s) + Q(t)$: a *coproduct* of position sets (not a product). This is the categorical product of the underlying polynomial functors.

7.1 Definition: `Container.prod`

Definition 7.1 (Product container)**.** The **product** $C \times D$ of containers $C = (S \triangleleft P)$ and $D = (T \triangleleft Q)$ is the container

$$(S \times T) \triangleleft ((s, t) \mapsto P(s) + Q(t)).$$

Note: the positions are the *sum* (coproduct) $P(s) + Q(t)$, not the product $P(s) \times Q(t)$. This is the standard polynomial product.

Lean.
```
def Container.prod (C D : Container) : Container where
  Shape := C.Shape 𝔼 D.Shape
  Pos   := fun st ⇒ C.Pos st.1 .+. D.Pos st.2  -- coproduct of
     position types
```

Haskell.
```
-- Product container: shapes are pairs, positions are sums.
prodPos :: (s → p) → (t → q) → (s, t) → Either p q
prodPos posC posD (s, t) = Left (posC s)  -- or Right (posD t)
-- i.e. Pos (s, t) = Either (Pos_C s) (Pos_D t)
```

Fidelity note. The product of polynomial functors (containers) uses a *coproduct* of position types because the polynomial functor associated to C at a set X is $\sum_s X^{P(s)}$, and the product of two such functors is $\sum_{(s,t)} X^{P(s)+Q(t)}$ (by the exponential law $X^{A+B} \cong X^A \times X^B$). This is distinct from the "set-level" product which would use $P(s) \times Q(t)$ for positions.

7.2 Definition: `Container.fst`

Definition 7.2 (First projection)**.** The **first projection** $\pi_1 : C \times D \to C$ has shape map $\mathrm{fst} : S \times T \to S$ and position map $\mathrm{inl} : P(s) \to P(s) + Q(t)$ (the coproduct injection, going *backwards* in position variance).

Lean.

```
def Container.fst (C D : Container) : ContainerMorphism (C.prod D) C
    where
  onShapes := Prod.fst
  onPos    := fun _ ⇒ Sum.inl
```

Haskell.

```
fstMorph :: Morph (s, t) (Either p q) s p
fstMorph = Morph fst (\_ → Left)
-- onPos sends (Left p) back to p; the backward map is Sum.inl
```

7.3 Definition: `Container.snd`

Definition 7.3 (Second projection)**.** The **second projection** $\pi_2 : C \times D \to D$ has shape map $\mathrm{snd} : S \times T \to T$ and position map $\mathrm{inr} : Q(t) \to P(s) + Q(t)$.

Lean.

```
def Container.snd (C D : Container) : ContainerMorphism (C.prod D) D
    where
  onShapes := Prod.snd
  onPos    := fun _ ⇒ Sum.inr
```

Haskell.

```
sndMorph :: Morph (s, t) (Either p q) t q
sndMorph = Morph snd (\_ → Right)
```

7.4 Definition: `ContainerMorphism.pair`

Definition 7.4 (Pairing)**.** Given $f : R \to C$ and $g : R \to D$, their **pairing** $\langle f, g \rangle : R \to C \times D$ has

$$\begin{aligned} \langle f, g \rangle_{\mathrm{shapes}}(r) &= (f_{\mathrm{shapes}}(r),\ g_{\mathrm{shapes}}(r)), \\ \langle f, g \rangle^\sharp_r &= [f^\sharp_r,\ g^\sharp_r] : P(f(r)) + Q(g(r)) \to R.\mathrm{Pos}(r). \end{aligned}$$

The position map case-splits via $[-,-]$ (eliminator for the sum).

Lean.

```
def ContainerMorphism.pair {R C D : Container}
    (f : ContainerMorphism R C) (g : ContainerMorphism R D) :
    ContainerMorphism R (C.prod D) where
  onShapes := fun r ⇒ (f.onShapes r, g.onShapes r)
  onPos    := fun r ⇒ Sum.elim (f.onPos r) (g.onPos r)
```

Haskell.

```
pairMorph :: Morph r rp s p → Morph r rp t q
          → Morph r rp (s, t) (Either p q)
pairMorph (Morph u g) (Morph v h) = Morph
  (\r → (u r, v r))
  (\r → either (g r) (h r))
```

7.5 Theorem: `ContainerMorphism.pair_fst`

Theorem 7.5 (Left computation rule for pairing)**.**

$$\langle f, g \rangle \circ \pi_1 \;=\; f.$$

Holds definitionally (`rfl`).

Lean.

```
theorem ContainerMorphism.pair_fst {R C D : Container}
    (f : ContainerMorphism R C) (g : ContainerMorphism R D) :
    (f.pair g).comp (Container.fst C D) = f
```

Haskell.

```
-- compMorph (pairMorph f g) fstMorph = f
```

7.6 Theorem: `ContainerMorphism.pair_snd`

Theorem 7.6 (Right computation rule for pairing)**.**

$$\langle f, g \rangle \circ \pi_2 \;=\; g.$$

Holds definitionally (`rfl`).

Lean.

```
theorem ContainerMorphism.pair_snd {R C D : Container}
    (f : ContainerMorphism R C) (g : ContainerMorphism R D) :
    (f.pair g).comp (Container.snd C D) = g
```

Haskell.

```
-- compMorph (pairMorph f g) sndMorph = g
```

7.7 Theorem: `ContainerMorphism.pair_eta`

Theorem 7.7 (Universal property of product (η-rule))**.** Every morphism $\chi : R \to C \times D$ is the pairing of its composites with the projections:

$$\langle \chi \circ \pi_1,\; \chi \circ \pi_2 \rangle \;=\; \chi.$$

Together with the two computation rules, this is the full universal property: $\mathrm{Hom}(R, C \times D) \cong \mathrm{Hom}(R, C) \times \mathrm{Hom}(R, D)$ naturally in R.

Lean.

```
theorem ContainerMorphism.pair_eta {R C D : Container}
    (chi : ContainerMorphism R (C.prod D)) :
    (chi.comp (Container.fst C D)).pair
      (chi.comp (Container.snd C D)) = chi
```

Haskell.

```
-- pairMorph (compMorph chi fstMorph) (compMorph chi sndMorph) = chi
```

7.8 Theorem: `ContainerMorphism.pair_unique`

Theorem 7.8 (Uniqueness of the pairing mediating morphism)**.** For every cone $(f : R \to C,\ g : R \to D)$, there is a *unique* morphism $\chi : R \to C \times D$ with $\chi \circ \pi_1 = f$ and $\chi \circ \pi_2 = g$, namely $\chi = \langle f, g \rangle$.

Lean.

```
theorem ContainerMorphism.pair_unique {R C D : Container}
    (f : ContainerMorphism R C) (g : ContainerMorphism R D)
    (chi : ContainerMorphism R (C.prod D))
    (h1 : chi.comp (Container.fst C D) = f)
    (h2 : chi.comp (Container.snd C D) = g) :
    chi = f.pair g
```

Haskell.

```
-- If compMorph chi fstMorph = f and compMorph chi sndMorph = g
-- then chi = pairMorph f g
```

8 Translation conventions

The following conventions supplement those in `Basic.tex`.

1. **Definitional equality.** Many laws in this file hold *definitionally* (the proof is `rfl`). This relies on Lean's η-rules for functions and structures. In Haskell, equational laws are asserted as comments; they would be QuickCheck properties in practice.
2. **HEq and transport.** The extensionality theorem `ContainerMorphism.ext` uses heterogeneous equality (`HEq`) because the position-map type depends on the shape map. The pointwise variant `ext'` uses an explicit transport ($\triangleright$). Neither has a direct Haskell analogue.
3. **Product positions are sums.** The product container has positions $P(s) + Q(t)$. This is structurally dual to the coproduct (where shapes are sums and positions are case-wise), and reflects the polynomial-functor formula $X^{A+B} \cong X^A \times X^B$.
4. **Variance teaching.** The injections ι_C, ι_D are identity-on-positions; the projections π_1, π_2 carry coproduct injections backward on positions. This duality captures the whole variance of containers at the level of (co)limits.